

Loop Engineering

不只是让 Agent 多跑几次，
而是建立可验证的工程系统

—— 分享人：碳硅同传



01

什么是 LOOP

重复不是工程，
停止条件才是边界

重复工作周期

采集上下文

采取行动

检查结果

必要时继续 ↓

停止条件 (边界)

✓ 目标达成

🚩 任务结束

⊘ 用户取消

🔋 资源上限

工程系统

触发

状态

验证

停止

恢复

人类判断

重复调用 ≠ 工程化 Loop

“ 官方定义

“一直重复工作周期，直到满足 **停止条件**”

重复调用



只是增加执行轮次
缺乏系统层面的边界控制

工程化 Loop

系统级连接：

触发

状态

恢复

人类判断

单轮内部机制



采集上下文



采取行动



检查结果



必要时继续



质量依赖

由 **代码库、文档、工具、验证与审查** 共同决定，而非循环次数



明确的停止边界
目标达成等条件触发

Loop 的完整定义：周期与停止条件缺一不可

Anthropic Claude Code 团队给出的原始定义

“agents repeating cycles of work **until a stop condition is met**”

[官方原文出处](#)

重复工作周期

- 01 采集上下文
- 02 采取行动
- 03 检查结果

必要时继续循环，进入下一轮

明确的停止边界

任务生命周期结束

目标达成

任务结束

用户取消

强制中断条件

需要新上下文

达到资源上限

四个判断维度

触发

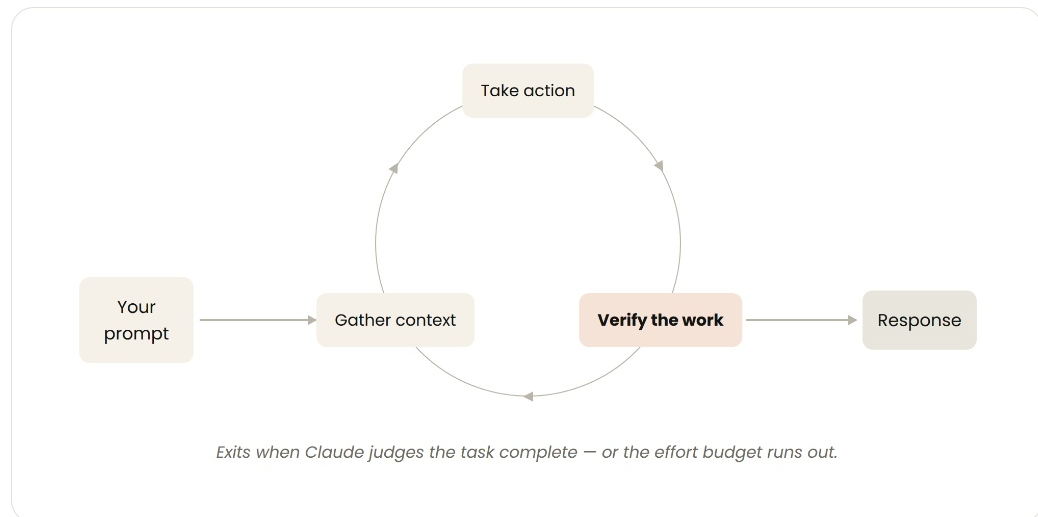
停止

primitive

任务类型

单轮 Turn-based 回路：检查之后才停止或继续

每条用户 prompt 开启一轮，用户查看后决定下一轮



👤 明确的停止边界

目标达成

任务结束

主动取消

新上下文

资源上限

1

读取上下文

由每一条用户 prompt 开启手动回路。

2

采取行动

Claude 利用工具执行修改与操作。

3

检查结果 控制节点

行动后必须运行检查验证，修正后才形成可交回的最终结果

4

停止或继续

将结果交回用户查看，由用户判断并决定是否发起下一轮。



工程化升级方向

将反复验收过程写入 **SKILL.md**，让 Agent 利用前端环境或工具来确认状态改变，使检查步骤可见、可测、可操作。

稳定 Loop 的九个最小要素

不是九项清单，而是三个彼此衔接的功能簇

≡ 稳定 Loop 不是单一 prompt，而是 九个互相约束的最小要素



Loop 的事实边界：三道门之后才谈可靠



边界结论 | Loop 仍可能因无效评估、错误目标或资源耗尽失败

01

第一道门：Outcome

核验最终环境是否正确，确认产物能否真正被使用。

✓ 产物可用标准

02

第二道门：Verification

独立运行任务、保存轨迹、记录产物并调用 Grader。

🔧 **Evaluation Harness**
运行任务并执行评测

🧰 **Agent Harness**
组织模型、工具与上下文

03

第三道门： Stop/Escalate

单一信号不足时停止或升级。

🚫 拒绝单独作为可靠证明：

- 模型自报完成
- 合法的 JSON 格式
- 好看的截图产出
- 单人审阅的认可

系统边界

三关连续成立才进入“可接受”

Loop 质量取决于：代码库 / 文档 / 工具 / 验证与审查

02

四类 Loop： 控制权如何从逐轮交互 走向长程运行

Turn-based

用户逐轮发起、检查与继续

Goal-based

可验证目标 + 最大轮数

Time-based

时间或外部状态变化触发

Proactive

组合调度、目标与权限网络

四类 Loop 的五维对照

对照维度	触发	停止	适用任务	primitive	资源控制
Turn-based	用户实时 prompt	完成或需要更多上下文	短任务、探索、非固定流程	普通 agentic loop	具体 prompt；把人工检查编译成 skill
Goal-based	用户实时设定目标	目标达成或达到最大轮数	有可验证退出条件的任务	/goal	明确完成标准与尝试上限
Time-based	固定时间间隔	人工取消或外部任务完成	周期工作；等待 PR、CI、队列变化	/loop、/schedule	拉长间隔；优先事件触发
Proactive	事件或时间调度	单任务达标退出；例行程序人工关闭	bug、issue 分诊、迁移、依赖升级等重复流程	schedule / goal / dynamic workflows / auto mode	小快模型跑常规，强模型做判断

从 Turn-based 到 Proactive：控制权外移，责任不退场

↻ Turn-based

用户逐轮发起、
检查并决定继续

🎯 Goal-based

以可验证目标和
最大轮数约束停
止

🕒 Time-based

由时间间隔或外
部系统状态触发

⚡ Proactive

组合调度、目
标、工作流与权
限，支持长程运
行

👤 责任边界

控制权外移，人不退场

🟡 任务启停

单任务要有明确退出条件，例行
程序必须支持随时由人类停止。

⚖️ 升级判断

涉及品味、风险、战略方向及赋
予新权限时，必须升级给人来判
断。

Turn-based: 用户是每一轮的实时调度器

用户触发，Claude 完成本轮，人决定是否继续

1

用户触发

起点

每条用户 **prompt** 开启一个手动回路，用户是实时调度器

2

Claude 执行簇

核心过程

读取上下文 → 执行修改 → 运行检查

3

结果交回

状态返回

Claude 返回它认为可用的阶段性结果

4

用户查看结果，判断走向

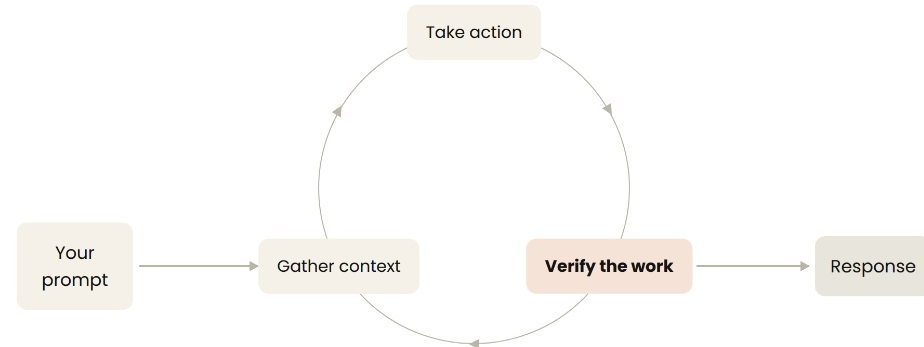
续跑

再发 prompt 开启下一轮

停止

判断任务完成，或缺上下文

回路官方结构



Exits when Claude judges the task complete — or the effort budget runs out.



适用范围与边界

适合 **短任务**、**探索** 和 **非固定流程**。

因为任务本身需要逐轮判断，该模式可让用户 **保留下一次** 的触发权。

Turn-based 的实践升级：把人工验收编译进 Skill

从重复检查到可试验、可纠错的自验闭环

传统 workflows

1. 用户提出修改要求



2. Agent 生成并修改代码



3. 人工反复干预

打开浏览器 → 点击交互 → 查控制台 → 发现失败要求重来

核心机制

Verification Loop

检查

Check



尝试修复

Fix



再检查

Re-check

验证确定性信号

- ✓ 基础：类型检查 / Linter / 测试 / 运行错误
- ✓ 前端：启动服务 → 真实浏览器交互 → 确认状态变化 → 查控制台

编译进 Skill 范式

把高频人肉验收固化为
SKILL.md



Agent 用工具操作并自动验证真实结果



✓ 失败后自动触发从头重验



生效条件与扩展

Skill 必须用新任务试运行：观察是否能准确触发、发现问题并纠错。验证通过后，再决定是否嵌入流程、串联更多步骤或推广至团队。



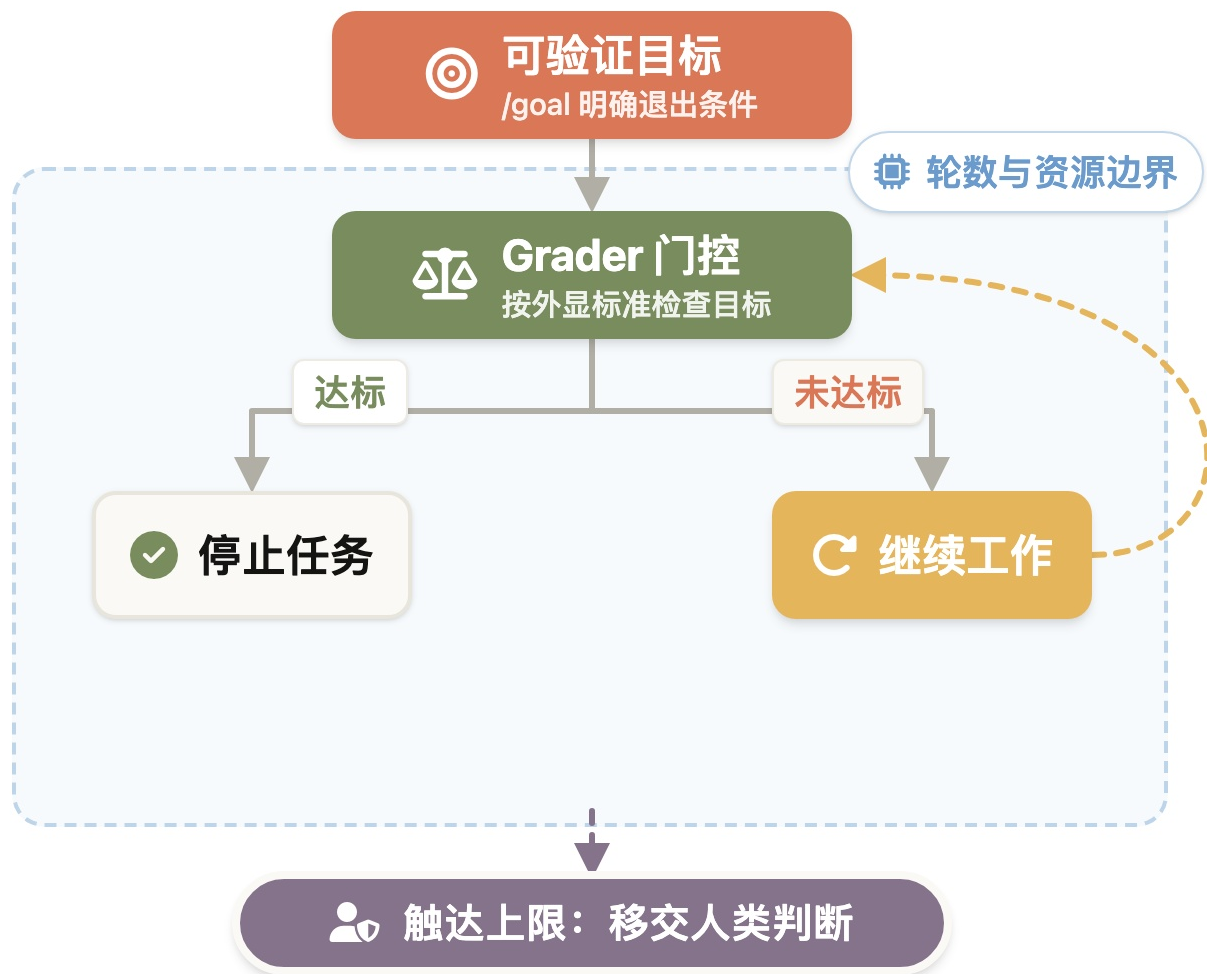
责任边界交接

Agent 负责：基于明确理想程序的前置自验。

人类保留：审美判断、真实交互意图确认，以及不可逆变更的最终决策权。

Goal-based: 把“完成”写成可验证合同

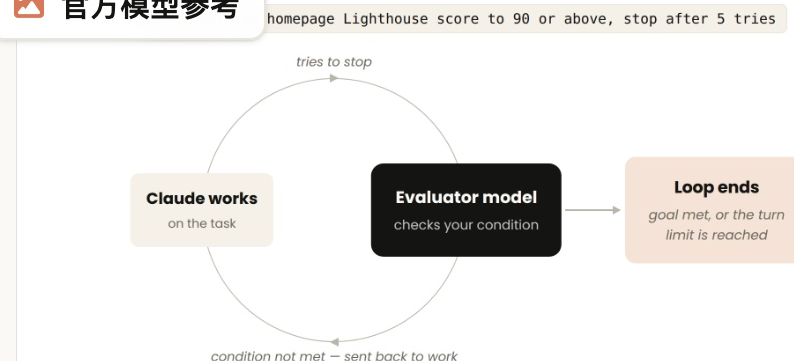
目标 → grader → 继续 / 停止，并把轮数、资源与人类判断写成边界



完整合同核心要素

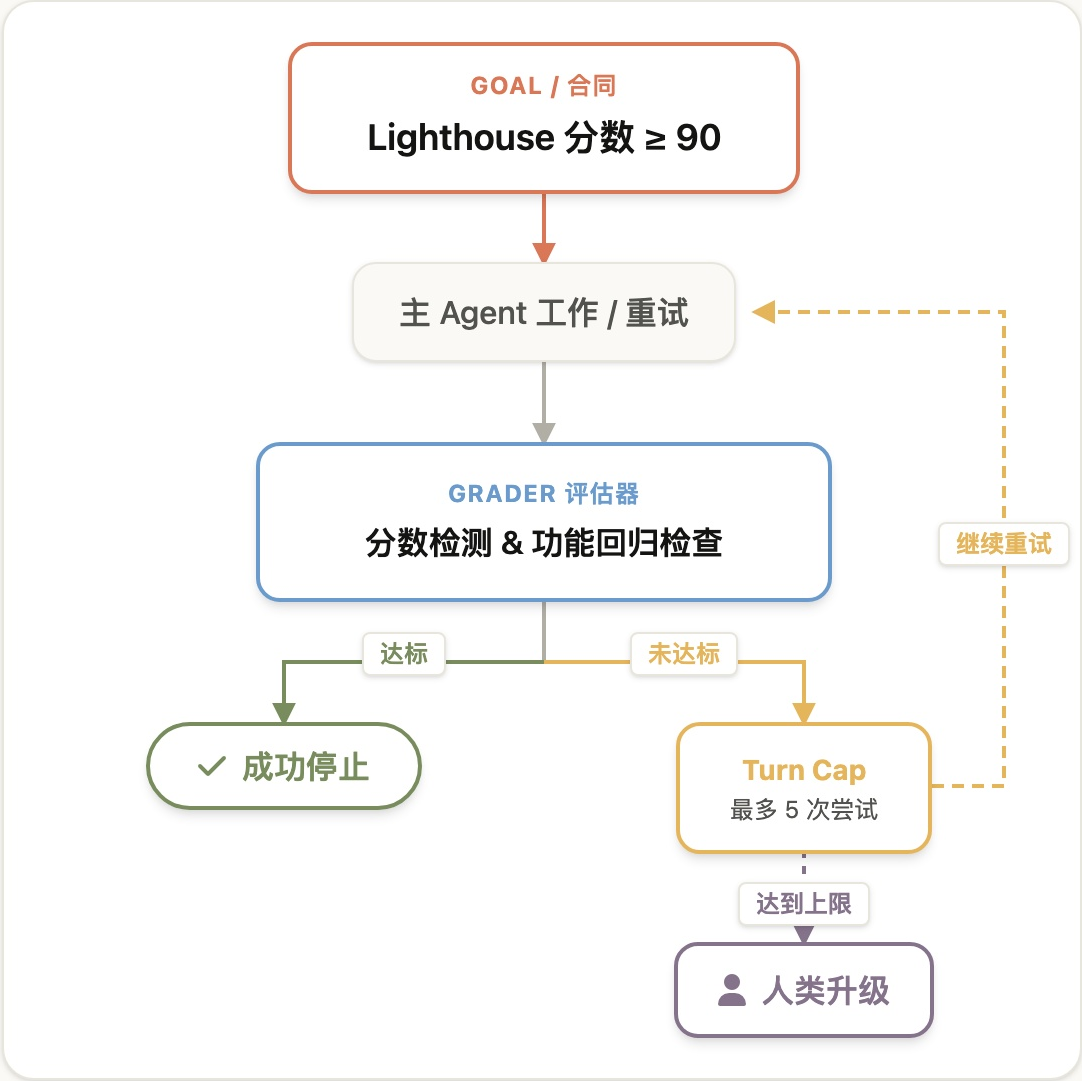
- **控制边界：**必须写清失败后允许修改的范围、最大重试轮数与资源耗用上限。
- **外显标准：**Grader 判定必须基于客观数据（如测试通过数、Lighthouse 门槛、无溢出数量），避免模型自评。

官方模型参考



Goal 控制图：把“完成”写成可验证的停止合同

目标 → grader → 重试 → turn cap → 人类升级



01 目标与 Grader

明确被验证的结果（如示例要求 Lighthouse 分数达 90分以上）。主 Agent 尝试结束时，由 **grader** 检查目标是否达标以决定停止或继续。

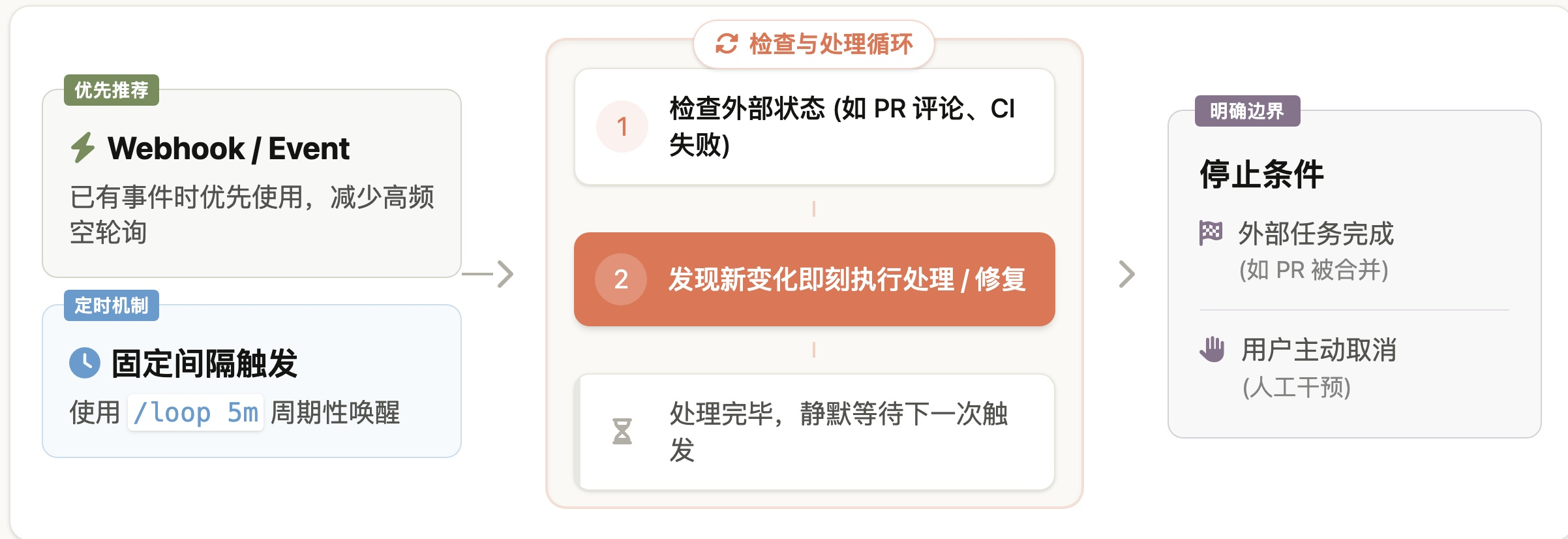
02 重试与回归门

grader 未通过就返回重试，不靠 Agent 自行判断“差不多了”。每轮除了重测分数还要检查**功能回归**，避免单一分数优化伤害真实产品体验。

03 上限与人类升级

turn cap（最大轮数或资源上限）必须预先写入合同；官方示例最多尝试 5 次。达到上限仍未通过或需判断时，就把决定交回人类。

Time-based: 把外部变化变成有边界的定时触发



本质洞察: 交出去的仅仅是「触发器」

核心是把“何时再次检查”交给时间。而停止的边界依然由外部系统状态或用户严格界定, 避免将其误用为无休止的持续自治。

Time-based: primitive 与边界

 /loop

运行位置

本机定时运行



停止边界

 本机关闭即停止

 /schedule

运行位置

转为云端 routine



停止边界

 人工取消或外部任务完成

 webhook
/ event

触发关系

可用时**优先事件触发**



核心原则

 服从外部节奏，避免空轮询

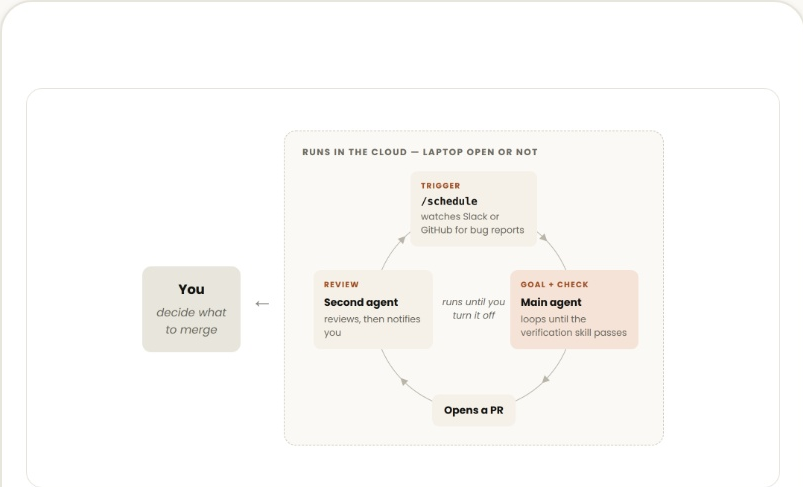
 可用性确认： 相关 primitive 的支持情况，必须按实际产品界面与官方文档进行确认。

Proactive: 五类能力如何组合成长程系统



i 运行逻辑本质

Proactive 的关键不在于让 Agent 多跑几轮，而是将多个 primitive 组合成**无需实时指挥的长程系统**。但不等于无限自治，例行程序必须保留人工停止机制。



实例：用户反馈闭环



Proactive 的权限边界



单任务：达标即退出

明确目标状态，任务完成后系统主动结束执行。



Routine：持续至人工关闭

针对重复工作流，例行程序将保持运行状态。

bug 报告

issue 分诊

迁移与依赖升级

⚡ auto mode

仅负责降低长程执行中的权限中断，不授予无限自治

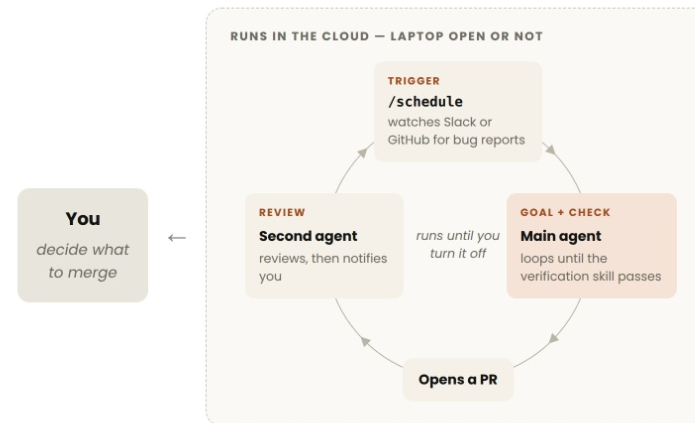


强制升级门：人类判断权

任何触及核心边界的决策，必须中止并交由人类确认：

⚠ 高风险修复 🎨 品味判断 📊 战略决策 🔑 新权限

官方架构对照表



工程边界： Primitive 实际可用性无法预设，需以当前产品界面、地区与文档为准。

03

Verification Loop: 把人工验收变成 纠错闭环

把原本依赖人工判断的过程，改造成**检查**、**修复**、**再检查**的自动闭环，让人工经验转化为可重复调用的系统能力。

[探索官方参考指南 →](#)



Verification Loop：把检查、修复和重验连成闭环

回到第一步重验

🔍 执行检查 (Check)

确定性信号（报错/测试）直接进入回路；代码工具无法推断的部分仍需人工验收。

真实验证拆解（前端示例）



出口验证



✖ 失败与修复 (Fix)

失败后并非修完就算，必须让 Agent 真正看到、测量并操作运行结果。

🌐 验证边界

步骤写进 `SKILL.md` 不等于能力成立。验证 Skill 必须在新任务样本上试验，观察其是否能成功触发检查并推动纠错。

📌 技能成熟演进路径

- 1 从近期最常手工执行的后续检查开始
- 2 用自然语言写出理想验证程序
- ✓ 在新任务观察效果 → 嵌入 / 串联 / 团队化



官方参考来源

Building Verification Loops in Cloud...



确定性信号与人工验收：自动化边界在哪里

可脚本化的确定性检查

适合交给代码工具反复执行、自动闭环的环节：

 类型检查

 Linter

 单元测试

 Runtime Error

 **工具能力边界：**让 Agent 通过真实工具 看到、测量、操作 结果，将反复验收写进 SKILL。

分界线：能否自动推断

仍需判断的语义验收

工具测不到全部质量，以下维度自动化到此收住

- 品味评估 (Taste)

- 交互意图 (Intent)

-  不可逆变更 (Irreversible)

-  **人工验收守区：**工具无法自动推断的部分，依然要由人来提供最终判断。

可靠性边界：勿轻信单一信号

 模型自称完成、JSON 合法、甚至一个审阅者认可，都只能覆盖部分问题。确定性检查和语义验收，必须分工配合。

 查阅官方资料

四种 Verification Skill 落点：触发方式决定适用边界

落点	触发方式	适用场景	限制与边界
Standalone	 人主动调用	跨 workflow、非每次必跑	 仍依赖人记得触发
Embedded	 自动执行	放进产生型 skill 内，适合具体 workflow	 只能编辑自己可控的 skill
Chained	 一个结束时调用另一个	串联经验证交接，形成端到端链路	 灵活性降低、token 成本可能增加
On every PR	 每次 PR 自动触发	适合团队级、已被证明稳定的验证	 不应过早推广尚未稳定的检查

 成熟路径：升级决策收束

[官方参考博客](#)

01 / 起步

高频手工后续检查

>

02 / 验证

新任务中观察触发与纠错

>

03 / 铺开

嵌入、串联或团队级使用

验证能力的成熟路径：先试验，再扩大触发

从高频手工检查长成团队级自动触发，关键是不能跳级



参考来源：Claude 官方博客验证实践

前端按钮案例：把真实交互变成可重复验收

从浏览器操作到失败重验，自动化接管重复检查，人保留语义判断



Agent 行动

新增按钮
读改代码并测试



verification skill

真实验收链 (接管重复动作)



启动服务



浏览器点击



确认状态



检查控制台

任一步失败，均从起点重新走完整验证



自动化边界

人工保留判断

品味、交互意图与不可逆变更，仍交由人类把控。

官方来源



Building verification loops
in Claude Code

claude.com/blog

04

跨会话继续工作： 外部真源是状态桥

核心参考资源：



Effective Harnesses for Long-Running Agents



Harness Design for Long-Running Apps

状态桥执行链路

状态断点

只做 context compaction 不足以保证稳定继续

初始化

先拆解特性清单，准备工作环境

单步执行

每次只推进一小步，离开前将状态明确落盘

外部记忆层

init.sh、进度文件、特性清单与 git 历史

恢复检查

新会话重读状态真源，执行端到端检查

交接标准

保持环境干净、可继续、可回滚

Context compaction 不等于跨会话状态恢复

真正的连续性来自可读取的外部真源

⚠ 单点局限

仅依赖 Context Compaction

不足以保证长程 Agent 稳定继续。它只能压缩当前窗口的信息，无法跨越会话生命周期。



离散会话断点
新会话可能遗忘
上一次的操作

📁 外部真源恢复

读取磁盘外部状态

- 📁 读取工作目录 (Worktree) 与进度
- 🔗 获取 Git 历史与特性状态
- ✅ 执行基本端到端 (E2E) 检查

初始化边界

不在首会话试图完成全部功能。首要任务是拆分特性清单并建立工作环境。

状态传输层

这些产物不是附属文档，而是 Loop 跨越窗口生命周期的核心传输依据。

🔗 来源: Anthropic Engineering

外部记忆层：四类产物如何共同承接状态



init.sh

与进度、特性清单、git 历史共同构成可恢复的外部记忆



progress

供新会话读取已有进度，接回留在磁盘的状态



feature list

承接需求拆分与特性状态，供恢复时检查



git history

保留工作历史，是新会话恢复前要读取的外部真源

品 不是附属记录，而是 **Loop** 的状态传输层



交接机制

后续 Agent 每次只推进一个小步骤，并在离开前把状态留在磁盘。



硬性边界

每一轮都必须留下 **干净、可继续、可回滚** 的状态，严禁把未完成的混乱环境直接交给下一会话。



参考来源

Effective Harnesses for Long-R...

Anthropic Engineering Blog

跨会话状态泳道：从初始化到 Resume Check

📖 参考资料

01 初始化



把需求拆成**特性清单**并建立工作环境

🚫 拒绝在首个会话试图一次性写完所有功能

02 单步执行



进入执行阶段后，后续 coding Agent 每轮只完成一个小步骤。



03 状态交接

离开前把状态写到磁盘，共同承接可恢复的外部记忆：

>_
init.sh

≡
进度文件

≡
特性清单

📁
git 历史

04 Resume Check

先读取

工作目录、进度、git 历史和特性状态



再检查

做基本的端到端检查，确认环境可继续



交接质量门 (硬门槛)

每一轮必须留下 **干净**、**可继续**、**可回滚** 的状态

↺ 回到下一轮初始化

Planner、Generator、Evaluator：把长任务拆成可控块

三段职责组成可评分、可修改、可交接的工作链



核心逻辑：长任务结构化传递

将复杂任务拆解，用 **结构化产物** 传递上下文，使拆解与交接成为明确的工作关系。

 [官方参考 →](#)

职责 01

Planner

拆分大任务，并定义块与块之间的**交接标准**与后续路由。

定义交接后
进入当前小块

职责 02

Generator

聚焦执行，**只在当前小块内**独立生成最终结果。

结果进入独立检查

职责 03

Evaluator

独立检查生成结果，指出失败项并触发修改。

 可评分标准 >  失败项 >  触发修改

Evaluator 核心边界

不能只给笼统的好评或差评，主观品质必须转成 **可判分维度**。

携带上下文

恢复合同： 每轮都留下可继续的工作环境

跨会话交接的三道门：干净、可继续、可回滚

离开前检查 (Exit Check)

必须同时满足



环境干净
无遗留未跟踪的临时文件或冲突



状态可继续
进度明确，下一步任务清晰可读



历史可回滚
操作已提交，可随时撤销至安全点

外部记忆

状态传输层

init.sh

进度文件

特性清单

git 历史

即使窗口结束，Agent 仍可从以上**外部真源**恢复并继续后续工作

↔

新会话恢复顺序 (Resume Check)

1

读取状态
加载工作目录及
四项外部记忆

2

端到端检查
执行基础验证
确保环境无异常

3

确认起点
重新对齐当前工作
与目标的一致性



交接禁区 (Boundary)

绝对禁止将未完成的混乱环境直接交给下一个会话，必须先整理状态。



参考官方设计

Harnesses for Long-running



评测回路

评测 Loop: 过程合理，不等于结果 正确

单凭模型自报完成或截图认可只是单一信号。必须将过程证据与结果证据分离，并引入多层级门禁验证。

[探索 Anthropic 评测指南](#)



Trajectory 不是 Outcome: 过程证据与结果证据要分开

路径解释发生了什么，结果验收确认是否真的做对



过程证据

TRANSCRIPT / TRAJECTORY



记录 Agent 的思考过程、工具调用与中间经历，还原操作链路。



能解释执行路径发生了什么，即解答“它怎么做”的问题。

≠



结果证据

OUTCOME



检查最终系统或环境是否处于正确状态，目标是否真正达成。



检查交付产物是否切实可用，即检验“最后成了什么”。



评测必须把过程合理与结果正确分开验收

过程路径合理不等于最终结果正确，单一信号不足以证明整个 Loop 可靠。



Anthropic 评测指南

两套 Harness：运行与评测分开

运行系统不能兼任唯一裁判



Agent Harness

≡ 组织模型、工具、权限和上下文

▶ 承担任务的实际运行

⊘ 不可合并为“Prompt 评分”



Evaluation Harness

✎ 运行评测任务并保存 Trace 轨迹

📋 记录产物并调用独立 Grader 裁判



独立门禁

↺ 失败样本进入回归集

结构与状态检查



功能验收



独立 Grader



人类抽样 / 高风险升级



Harness Design for Long-running Apps

www.anthropic.com



Demystifying Evals for AI Agents

www.anthropic.com

评测分层门禁：五道门，不靠单点评分

结构、功能、语义、人工与回归，各自捕获不同问题

01

确定性检查

先验证结构与状态，把可明确判定的问题挡在第一道门

02

功能验收

再检查最终结果是否真正通过功能层面的验收

03

独立 grader

当判断依赖品味或语义时，引入独立评价

04

人类抽样与高风险升级

保留人工判断，并承接高风险案例

05

失败回归化

进入回归集，继续检查后续修复是否损害其他案例



共同边界

没有任何单一评测层，能单独证明整个 Loop 可靠。

重点不是把所有判断压成一个总分，而是让不同层各自承担能验证的责任。



参考来源

Demystifying evals for AI agents

[Anthropic Engineering](#)

Lighthouse Goal-based: 用阈值和尝试上限定义停止

90 分目标门 + 功能回归门 + 最多 5 次 + 人类升级



单一分数不能替代真实产品体验，需保留回归门和人判决定退出。

[参考案例](#)

失败样本回归化：修复当前案例，同时检查是否损害其他案例



06

从生产失败 到系统改进

让每次纠错进入下一轮



参考案例

building-self-improving-tax-agents...

correction → trace

- 1 专业人员纠正真实错误，系统保存输入与最终更正。

finding → eval

- 2 错误被分类，并编成可重现的评测（eval）。

fix

- 3 Agent 在有边界工作区中进行调查并修复。

targeted + regression

- 4 候选修复通过多项评测，再由人类确认推进。

进入下一轮系统改进

- 5 失败规则被编进系统，避免再次发生。

跨厂商共同基础： Agent 可读环境、可执行约束与反馈回路

🔗 参考：Harness Engineering



生产改进飞轮：correction 到 regression

从专家纠正到双重评测，再由人确认推进



自改进的核心本质

非模型无约束自修改，而是将生产错误转化为受控可追溯的工程迭代任务

[查看案例参考](#)

用户反馈如何进入 Proactive 处理回路



生产级 Loop：资源治理不是附加项

六个控制项，把选择、停止、试跑、脚本化、频率和 usage 连成一套运行约束

 参考：harness-engineering

01 启动前决策

模型 / primitive

按任务复杂度选择，**不默认多 Agent 或长 Loop 架构。**

完成 / 停止条件

执行前明确定义**何时完成**、**何时必须结束退出。**

02 批量前固化

小样本试跑

在进入大批量自动化执行前，**必须先进行小规模验证。**

脚本化固化

确定性的工作交由脚本处理，**不在每一轮重新推理。**

03 运行中约束

频率控制

避免盲目高频轮询，检查频率**需匹配外部状态**的变化节奏。

Usage 检查

持续监控消耗，将模型调用与 effort 作为整体**成本杠杆**进行约束

Loop 的终点：把失败沉淀为系统能力



可靠性边界约束

⚠ 单一评测层（模型自检、JSON 合法、单次认可）不能证明整个 Loop 可靠。

🔒 自改进依赖专家纠正、生产证据、任务边界与评测门。

感谢观看，我是碳硅同传

下一期：Graph Engineering

[getting-started-with-loops](#)